

Hackathon Project Phases Template that ensures students can complete it efficiently while covering all six phases. The template is structured to capture essential information without being time-consuming.

Hackathon Project Phases Template

Project Title:

WriteWise: Essay and Assignment Feedback Tool using Bert and T5

Team Name:

Infinity League

Team Members:

- Arudala Swapna Sri Vidya
 - Iswarya Bandaru
 - Naga Trisha Andaluri
 - Arigela Sri Lakshmi naga vyshnavi
-

Phase-1: Brainstorming & Ideation

Objective:

- **Improve Writing Quality:** Help users enhance the clarity, coherence, style, and overall quality of their writing. This includes grammar checks, sentence structure improvements, and tone adjustments.
- **Provide Writing Suggestions:** Offer real-time suggestions for vocabulary, phrasing, and sentence flow to enhance the effectiveness of the written message.
- **Save Time and Effort:** Automate repetitive tasks like formatting, content editing, and proofreading to save users time, allowing them to focus on higher-level writing tasks.
- **Enhance User Experience:** Create an intuitive and user-friendly interface that makes AI-assisted writing accessible to both beginner and advanced writers.

Key Points:

- **Problem Statement:**

- In today's fast-paced digital world, effective communication through writing is essential for personal, academic, and professional success. However, many individuals struggle with various aspects of the writing process, such as grammar, sentence structure, clarity, creativity, and coherence. These challenges can result in subpar written content, wasted time spent on revisions, and hindered productivity. Furthermore, writers, especially non-native speakers and those learning new writing styles, may find it difficult to improve without constant access to expert feedback.

- **Proposed Solution:**

- To address the challenges faced by writers, WriteWise AI will be developed as a comprehensive AI-powered writing assistant that supports users in improving all aspects of their writing process. By integrating advanced natural language processing (NLP) and machine learning algorithms, the solution will offer an array of features designed to enhance writing quality, foster creativity, and provide personalized feedback for a wide range of writing tasks.

- **Target Users:**

- **Students (High School, College, University):**
- Professionals with specific features that meet the needs of different writing goals. Here are the key target user groups:
 - **Needs:** Assistance with academic writing, including essays, research papers, and reports. Support with grammar, structure, citations, and improving clarity.
- **Use Case:** WriteWise AI helps students refine their writing, ensuring it meets academic standards and is free from common language errors. It can also suggest improvements in argumentative writing, aid with thesis development, and check for proper citation formatting (e.g., APA, MLA).
- **Professional Writers (Content Writers, Bloggers, Journalists):**
- **Needs:** High-quality writing that is engaging, error-free, and optimized for SEO. Speed and efficiency in content creation are key.
- **Use Case:** WriteWise AI aids content creators in crafting well-structured, polished articles or blog posts. It offers SEO optimization, content suggestions, and style improvements, ensuring that writing is not only informative but also engaging and search-engine-friendly.

- **Expected Outcome:**

- **1. Improved Writing Efficiency:**
 - **Outcome:** Users will be able to write faster and more efficiently by leveraging AI-powered suggestions for grammar, structure, and style improvements.
 - **Measurable Goal:** Users should experience a 20-30% reduction in time spent revising their work.
 - **2. Enhanced Writing Quality:**
 - **Outcome:** WriteWise AI will assist in enhancing the clarity, coherence, and professionalism of written content by offering real-time grammar checks, sentence restructuring suggestions, and readability improvements.
 - **Measurable Goal:** A targeted 15-20% improvement in writing quality as assessed by automated readability tools and user feedback.
-

Phase-2: Requirement Analysis

Objective:

The primary objective of the WriteWise AI project is to enhance the writing process by utilizing advanced AI technology to assist in generating, improving, and refining written content. The project aims to achieve the following specific objectives:

1. Automate Content Creation:

- Develop and implement AI-driven tools to assist users in generating high-quality written content quickly and effectively, catering to various types of writing, such as articles, reports, blog posts, and more.

2. Improve Writing Quality:

- Utilize AI algorithms to suggest improvements in grammar, punctuation, structure, style, and tone, helping users produce polished and professional content consistently.

Key Points:

- **Technical Requirements:**

1. Programming Languages

- **JavaScript (Node.js):**

- If the AI tool includes a web-based interface or real-time collaboration features, JavaScript (with **Node.js** for server-side applications) would be essential. It can handle front-end interactions and back-end requests to the AI engine.

2. Frameworks and Libraries

- **TensorFlow / PyTorch:**

- These frameworks are essential for building and training deep learning models, especially in natural language processing (NLP). They would be used for tasks such as text generation, language understanding, and recommendation systems.

- **Hugging Face Transformers:**

- Hugging Face provides a comprehensive library for pre-trained NLP models like **GPT**, **BERT**, and others. This is invaluable for tasks such as text generation, summarization, or sentiment analysis.

3. Tools and Platforms

- **AWS (Amazon Web Services) or Google Cloud AI:**

- Cloud platforms like AWS or Google Cloud provide infrastructure for hosting models, databases, and APIs. They also offer managed services for machine learning (e.g., **AWS SageMaker**, **Google AI Platform**) that can simplify model training and deployment.

- **Functional Requirements:**

For **WriteWise AI**, the **functional requirements** should cover the core features and capabilities that will enable the system to deliver on its objectives and fulfill user needs effectively. Below are key features that the project must have:

Functional Requirements for WriteWise AI

1. Text Generation

- **AI-driven Content Creation:**

- The AI should be able to generate high-quality, coherent text based on user inputs or prompts. This feature is essential for users who need assistance in drafting articles, blog posts, or other written content.

- **Constraints & Challenges:**

- **1. Accuracy of AI Models**

- **Challenge:** While AI can perform well in basic language tasks, it may struggle with nuanced language, context, and tone.
- **Constraint:** Ensuring that WriteWise AI provides accurate grammar corrections, style suggestions, and content improvements without introducing errors or misunderstandings.
- **Solution:** Using sophisticated NLP models and training on large, diverse datasets to reduce misinterpretations and errors.

- **2. Context Understanding**

- **Challenge:** Understanding the context of a user's writing is crucial for delivering personalized and relevant suggestions.
- **Constraint:** AI may struggle to comprehend the full context of a document or specific writing style, especially when dealing with informal or specialized language.
- **Solution:** Implementing context-aware algorithms or deep learning models that can better understand the intent behind each piece of text.

- **3. Handling Diverse Writing Styles**

- **Challenge:** Users write in various styles, ranging from academic writing to casual content. WriteWise AI needs to adapt to different tones and formats.
- **Constraint:** A one-size-fits-all solution may not be suitable for all types of writing.
- **Solution:** Allow users to select or specify the type of writing (e.g., formal, casual, academic), so the AI can adjust suggestions accordingly.

Phase-3: Project Design

Objective:

- **1. Accuracy of AI Models**

- **Challenge:** While AI can perform well in basic language tasks, it may struggle with nuanced language, context, and tone.
- **Constraint:** Ensuring that WriteWise AI provides accurate grammar corrections, style suggestions, and content improvements without introducing errors or misunderstandings.
- **Solution:** Using sophisticated NLP models and training on large, diverse datasets to reduce misinterpretations and errors.

- **2. Context Understanding**

- **Challenge:** Understanding the context of a user's writing is crucial for delivering personalized and relevant suggestions.
- **Constraint:** AI may struggle to comprehend the full context of a document or specific writing style, especially when dealing with informal or specialized language.
- **Solution:** Implementing context-aware algorithms or deep learning models that can better understand the intent behind each piece of text.

- **3. Handling Diverse Writing Styles**

- **Challenge:** Users write in various styles, ranging from academic writing to casual content. WriteWise AI needs to adapt to different tones and formats.
- **Constraint:** A one-size-fits-all solution may not be suitable for all types of writing.
- **Solution:** Allow users to select or specify the type of writing (e.g., formal, casual, academic), so the AI can adjust suggestions accordingly.

Key Points:

1. **System Architecture Diagram:**

- **User Interface (Frontend)**

This is the front-facing part of the application where users type in text and interact with the system.

Components: Text Input Area, Buttons, Display Section.

Data Flow: User enters text → Sends text to the API.

- **API Layer**

Acts as a middle layer, receiving user input from the frontend and communicating with

the NLP Model.

Components: API endpoints for text submission and response.

Data Flow: Receives text → Sends to NLP Model for analysis → Returns processed results to frontend.

□ NLP Model

The core of WriteWise AI, processing the text input and generating suggestions for grammar, style, and readability.

Components:

Grammar Checker

Style Enhancer

Readability Scorer

Suggestions Generator

Data Flow: Processes text → Generates suggestions → Sends them to the API Layer.

□ Suggestions & Feedback

After text analysis, suggestions and improvements are generated, including grammar corrections, readability feedback, etc.

Components: Feedback Box displaying results.

Data Flow: Displays suggestions → Feedback to the frontend.

2. UI/UX Considerations:

1. Simplicity and Clean Design

Minimalistic Interface: Avoid clutter. A clean interface with essential elements will allow users to focus on the task at hand (writing and reviewing suggestions).

White Space: Proper use of white space makes the interface less overwhelming and easier on the eyes, ensuring users can navigate easily.

Typography: Choose a readable font, such as **Roboto** or **Arial**, with clear headings, body text, and distinct font sizes for emphasis. The text input area should also have a comfortable font size for readability.

2. Clear and Intuitive Navigation

Simple Navigation Structure: Keep the flow straightforward—users should easily understand where to type, how to submit the text for analysis, and how to access feedback.

Visible Buttons: Highlight primary actions (e.g., "Analyze Text", "Submit") using contrasting colors so they stand out.

Feedback Mechanism: Users should get instant feedback when they interact with the system (such as on text submission) to reduce frustration.

Multiple Writing Modes: If possible, offer different modes, such as "Basic" or "Advanced", depending on the user's needs (e.g., novice users might prefer simpler

suggestions, while advanced users may want detailed grammatical explanations).

Phase-4: Project planning(Agile Methodologies)

Objective:

- The **objective of project planning** is to ensure that a project is completed successfully by outlining clear goals, defining tasks, setting timelines, and allocating resources. It provides a structured approach to manage all aspects of the project from start to finish. Specifically, for **WriteWise AI**, project planning aims to:

1. Define Clear Goals and Scope

- Establish what the project aims to achieve (e.g., building a text analysis tool for improving writing quality).
- Clearly outline what features the system will have, such as text analysis, writing suggestions, user authentication, etc.
- Set boundaries to prevent scope creep (ensuring that the project doesn't expand beyond the original intent).

2. Ensure Efficient Resource Allocation

- Identify and allocate resources (e.g., team members, tools, technologies, budget) effectively.
- Assign roles and responsibilities so each team member knows their tasks and deadlines.
- Ensure that tools like NLP libraries and backend technologies are available for development.

Key Points:

1. Sprint Planning:

Sprint Planning for WriteWise AI involves defining the goals, tasks, and timelines for a specific iteration in the development cycle. A sprint usually lasts for 2 days, and sprint planning is a critical part of the **Agile methodology** to ensure teams stay focused on delivering high-value features.

Here's a comprehensive **sprint planning breakdown** for **WriteWise AI**:

1. Sprint Goal

Sprint Goal: Implement and enhance core functionality for WriteWise AI's text analysis, writing suggestions, and user interface.

2. Sprint Duration

Sprint Length: 2 days

Start Date: 11-02-2025

End Date: 12-02-2025

1. Set up Project Environment and Initial Structure

Priority: High

Time: 1 day

Set up version control (Git repository).

Set up basic file structure for front-end and back-end.

Define project dependencies (e.g., NLP libraries, server setup).

Create the project skeleton with HTML/CSS and a basic server structure.

2. Build Text Input and Display UI (Front-end)

Priority: High

Time: 1 day

Create a text input box for users to enter text.

Design an area to display feedback (sentiment analysis, grammar suggestions).

Implement UI responsiveness (mobile/tablet-friendly).

Ensure basic error handling for empty inputs.

phase-5: Project Development

Objective:

The **objective** of developing **WriteWise AI** is to create an intelligent writing assistant powered by artificial intelligence (AI) and natural language processing (NLP) technologies. The goal is to help users improve the quality and effectiveness of their writing by providing real-time feedback on various aspects such as grammar, sentence structure, style, and vocabulary.

Key Points:

1. Technology Stack Used:

Technologies used to build the user interface and enhance the user experience:

- **HTML5:**
 - To create the structure and layout of the web application (e.g., forms, buttons, navigation).
- **CSS3:**
 - For styling the page, providing responsive design, animations, and smooth transitions to enhance the user experience.
- **JavaScript (ES6+):**
 - To make the page interactive and dynamic. It handles UI-related interactions like submitting text, fetching analysis results, and presenting suggestions in real-time.

2. Development Process:

The development process of WriteWise AI involves several stages to ensure a smooth build, deployment, and iterative improvements of the product. The process is typically divided into phases: Planning, Design, Development, Testing, Deployment, and Maintenance. Here's an in-depth look at each stage:

1. Planning Phase

Objective: Define the goals, scope, and requirements of WriteWise AI.

Requirements Gathering: Understand the needs of the target audience (writers, content creators, students, etc.). Collect user feedback, research existing solutions, and determine what features WriteWise AI must offer (text analysis, writing improvement, suggestions, etc.).

Defining Features and Functionalities:

AI-based writing analysis (noun/verb counts, sentence structure analysis, etc.).

Grammar and style suggestions (e.g., active/passive voice, sentence clarity, word choice).

Seamless user interface for text input and feedback.

User registration, login, and personalized text history.

3. Challenges & Fixes:

1. Challenge: Accuracy of AI Suggestions

Problem: The primary challenge for WriteWise AI is ensuring the AI model delivers **accurate and meaningful suggestions** related to grammar, sentence structure, and word choice. Incorrect or irrelevant suggestions can frustrate users and lower the platform's value.

Fixes:

- **Improving AI Training:** Continuously train the AI models using large, diverse datasets of well-written content. For NLP tasks, fine-tune pre-trained models like **GPT-3** or **BERT** for more accurate text suggestions.
 - **Continuous Evaluation:** Regularly evaluate the AI's output with a feedback loop from users. Implement a system where users can rate suggestions, allowing the AI to learn from its mistakes and improve over time.
 - **Hybrid Approach:** Combine machine learning with rule-based logic for specific tasks (e.g., grammar rules, sentence construction). This ensures that the model can handle edge cases better while still offering flexibility for more complex writing patterns.
-

Phase-6: Functional & Performance Testing

Objective:

Testing is a crucial part of ensuring that WriteWise AI works as intended and delivers high-quality performance for its users. There are two primary types of testing that need to be conducted:

Functional Testing: Ensures that the features and functionalities of WriteWise AI work as expected.

Performance Testing: Evaluates how well WriteWise AI performs under various load conditions and helps identify areas for optimization.

Key Points:

1. Test Cases Executed:

1. Text Input Validation

Test Case ID: TC-01

Description: Verify that the text input field accepts different types of text.

Pre-condition: User is on the WriteWise AI homepage.

Test Steps:

Navigate to the text input field.

Input plain text (short, medium, long text).

Input special characters or numbers.

Expected Result: Text should be accepted, and no errors should occur.

2. Grammar Suggestion Feature

Test Case ID: TC-02

Description: Ensure the AI provides appropriate grammar suggestions.

Pre-condition: Text input is provided.

Test Steps:

Input a grammatically incorrect sentence (e.g., "She don't like cake.").

Click on the "Analyze Text" button.

Expected Result: The system should highlight the grammar error and suggest "She doesn't like cake." or a similar suggestion.

3. Spell Check Functionality

Test Case ID: TC-03

Description: Verify that the AI detects spelling mistakes in the text.

Pre-condition: Text input is provided.

Test Steps:

Input a sentence with intentional misspelled words (e.g., "I love the comuter.").

Click on the "Analyze Text" button.

Expected Result: The AI should highlight the misspelled word ("comuter") and suggest the correct spelling ("computer").

4. Noun/Verb/Sentence Count Feature

Test Case ID: TC-04

Description: Validate the AI's ability to correctly identify nouns, verbs, and sentences.

Pre-condition: Text input is provided.

Test Steps:

Input a sentence like "The quick brown fox jumps over the lazy dog."

Click on the "Analyze Text" button.

Expected Result: The AI should display counts of nouns, verbs, and sentences (e.g., Nouns: 5, Verbs: 1, Sentences: 1).

1. Bug Fixes & Improvements:

Bug Fixes

1. Grammar Detection Bug

- **Issue:** Grammar suggestions sometimes miss certain errors or suggest incorrect

changes.

- **Fix:** Improve the AI's grammar detection algorithm by integrating a more advanced Natural Language Processing (NLP) model, such as a transformer-based model like BERT or GPT-3, to better understand context and improve suggestions.

2. Spelling Suggestions Not Triggering

- **Issue:** Misspelled words are not always highlighted, or the correct suggestion does not appear.
- **Fix:** Enhance the spelling checker to detect a broader range of spelling errors and work across various contexts (e.g., names, place names, slang) with a more refined spell-check dictionary.

3. Slow Response Time on Large Text Inputs

- **Issue:** The system takes too long to process larger blocks of text or documents.
- **Fix:** Optimize the backend algorithms for processing large text inputs and leverage multi-threading or parallel processing for faster response times.

4. UI Glitches on Mobile Devices

- **Issue:** The user interface does not render properly on certain mobile devices (e.g., buttons overlapping, input fields cut off).
- **Fix:** Perform responsive design enhancements using media queries and CSS Grid/Flexbox to ensure the UI adapts smoothly to various screen sizes and resolutions.

2. Final Validation:

1. Functional Testing

Functional testing ensures that all features of WriteWise AI work as intended and meet the requirements outlined in the project.

Key Areas to Test:

- **Text Input Validation:** Verify that the AI can correctly receive and process text inputs of varying lengths and complexities.
- **Grammar and Spell Check:** Ensure that grammar and spelling suggestions are accurate and relevant.
- **Text Suggestions:** Confirm that the AI suggests improvements in style, clarity, and structure, offering meaningful suggestions based on the input.
- **Multilingual Support:** Test that the AI correctly handles different languages if the feature is implemented.
- **Authentication:** Validate the login/signup functionality to ensure smooth user experience when creating or logging into accounts.
- **User Profile:** Ensure that user preferences (e.g., tone, writing style) are saved and applied correctly.
- **Real-time Collaboration:** If enabled, test that multiple users can collaborate on the same document without errors or lag.

2. Performance Testing

Performance testing verifies that the WriteWise AI performs efficiently and can handle varying levels of load, user requests, and large-scale inputs.

Key Areas to Test:

- **Speed & Response Time:** Test how quickly the AI processes different types of content, including short sentences and large paragraphs, ensuring the response time stays within acceptable limits (ideally < 5 seconds for smaller texts).
- **System Scalability:** Validate how the system scales under heavy loads, with multiple users using the tool simultaneously. This can include testing server resources and how quickly the system can handle increases in traffic.
- **Concurrency:** Ensure that multiple users can use the system at the same time without causing degradation in performance.

- **Resource Usage:** Test memory, CPU, and network usage to ensure the system can run optimally, especially when processing large documents or high traffic.
-

Final Submission

1. Project Report (Based on the Templates):

Your project report should be a comprehensive document that details all aspects of your project, following the provided templates. It should be well-structured, easy to read, and free of grammatical errors and typos. Here's a suggested structure (adapt it to your specific template):

- **Executive Summary:** A brief overview of the project, its goals, and key findings. Write this last after you've completed the rest of the report.
- **Introduction:** Background information on the project, its purpose, and its scope.
- **Requirements Analysis (Phase 2):** Detailed description of the functional and technical requirements, constraints, and challenges.
- **Project Planning (Phase 4):** Explanation of the Agile methodologies used, sprint planning, task allocation, and timelines.
- **Project Development (Phase 5):** Description of the technology stack, development process, and challenges faced and their solutions.
- **Functional & Performance Testing (Phase 6):** Details of the test cases executed, bug fixes, improvements, final validation, and deployment information.
- **Conclusion:** Summary of the project outcomes, lessons learned, and future work.
- **Appendices (if applicable):** Include any supporting materials, such as detailed test results, code snippets (if short and relevant), or diagrams.

2. Demo Video (3-5 Minutes):

Your demo video should provide a concise and engaging overview of your project. It should demonstrate the key features and functionalities of your application.

- **Script/Storyboard:** Plan your demo video in advance. Write a script or create a storyboard to ensure a clear and logical flow.
- **Visuals:** Use clear and high-quality visuals. Show the user interface, key features, and any relevant data or diagrams.
- **Narration:** Provide clear and concise narration to explain what you're demonstrating. Keep it engaging and to the point.
- **Focus on Key Features:** Don't try to cram too much into the video. Focus on the

most important features and functionalities.

- **Target Audience:** Consider your target audience when creating the demo video. What are they most interested in seeing?
- **Editing:** Edit the video to ensure a smooth and professional presentation.

3. GitHub/Code Repository Link:

Your code repository should be well-organized and easy to navigate.

- **README File:** Include a clear and informative README file that explains how to set up and run the project. This is essential!
- **Code Comments:** Comment your code thoroughly to make it understandable.
- **Directory Structure:** Use a logical directory structure to organize your code files.
- **Version Control:** Use Git effectively to track changes and collaborate (if applicable).
- **License:** Include a license file to specify how others can use your code.

4. Presentation:

Your presentation should be a concise and compelling summary of your project. It's an opportunity to highlight the most important aspects of your work.

- **Key Points:** Focus on the key points of your project, including the problem you're solving, your solution, and the results you achieved.
- **Visuals:** Use clear and engaging visuals, such as charts, graphs, and screenshots.
- **Practice:** Practice your presentation beforehand to ensure a smooth and confident delivery.
- **Audience:** Consider your audience when preparing your presentation. What are they most interested in hearing?
- **Time Limit:** Adhere to the specified time limit for your presentation.

Tips for Success:

- **Start Early:** Don't wait until the last minute to work on your final submission. Start early and allocate sufficient time for each component.
- **Collaboration:** If you're working in a team, collaborate effectively and divide the work among team members.
- **Review:** Review all of your submission materials carefully before submitting them. Ask someone else to review them as well for a fresh perspective.

- **Follow Instructions:** Carefully follow the instructions provided for the final submission.
- **Professionalism:** Present your work in a professional and polished manner.

By following these guidelines, you'll be well-prepared to submit a high-quality final submission that showcases your hard work and the success of your project. Good luck!
